F1 AUTONOMOUS RACING AI

COMPREHENSIVE DEVELOPMENT GUIDE

By:- Pratham Patel

Target: Virtual Yas Marina Circuit (Abu Dhabi)

Hardware: Intel Core Ultra 9 275HX | RTX 5070Ti 12GB | 32GB RAM

1. Introduction & Project Overview 3
2. Understanding the Problem Domain 3
2.1 F1 2026 Regulations Summary 3
2.2 Yas Marina Circuit Characteristics 3
3. Technology Stack & Software Architecture 4
3.1 Recommended Simulation Platforms 4
Option A: Assetto Corsa with OpenAI Gym Interface (Recommended) 4
Option B: CARLA Simulator 4
Option C: F1/10 ROS Simulator 4
3.2 Core Software Components 4
4. Development Phases with KILO 5
Phase 1: Environment Setup (Week 1-2) 5
Phase 2: Vehicle Dynamics Model (Week 2-4) 5
Phase 3: Perception & State Estimation (Week 4-6) 6
Phase 4: Racing Line Optimization (Week 6-8) 6
Phase 5: Reinforcement Learning Agent (Week 8-12) 6
5. F1 Driver Knowledge Integration 7
5.1 Gear Shift Strategy 7
5.2 Throttle & Brake Modulation 7
5.3 Corner Execution 7
5.4 Pit Stop Strategy 7
5.5 Energy Management (2026 Regulations) 8
6. Training Strategy & Hyperparameters 8
6.1 Recommended Neural Network Architecture 8
6.2 Hyperparameter Guidelines 8
7. Evaluation & Continuous Optimization 9
7.1 Performance Metrics 9
7.2 Sim-to-Real Considerations 9
8. Effective KILO Usage for This Project 9
9. Project Timeline & Milestones 9
10. Key Resources & References 10
11. Getting Started Checklist 10

Note: Right-click the Table of Contents and select 'Update Field' to display correct page numbers.

1. Introduction & Project Overview

This guide provides a comprehensive roadmap for building an advanced autonomous F1 racing AI that can compete on the virtual Yas Marina Circuit (Abu Dhabi) with the goal of breaking lap records while adhering to all FIA Formula 1 regulations. The project leverages the KILO AI coding platform for accelerated development, combining reinforcement learning, classical control theory, and real-world F1 driving knowledge to create a competitive racing agent.

Your hardware configuration (Intel Core Ultra 9 275HX, RTX 5070Ti 12GB, 32GB RAM) is well-suited for this project, offering excellent computational power for both simulation rendering and neural network training. The RTX 5070Ti with 12GB VRAM provides sufficient memory for deep learning models, while the 32GB system RAM ensures smooth operation of multiple simulation instances.

2. Understanding the Problem Domain

2.1 F1 2026 Regulations Summary

The 2026 F1 regulations introduce significant changes that your AI must understand and comply with. These regulations emphasize a 50:50 power split between the internal combustion engine and electric motor, making energy management crucial for optimal lap times. The cars will be lighter, shorter, and more nimble, with active aerodynamics featuring a low-drag mode for straights and a high-downforce mode for corners. The Drag Reduction System (DRS) has been replaced with an 'overtake mode' that provides bursts of extra electric energy when within one second of another car.

2.2 Yas Marina Circuit Characteristics

The Yas Marina Circuit in Abu Dhabi features 16 turns across 5.281 km, with unique characteristics across three distinct sectors. Sector One includes high-speed corners at Turns 2 and 3, requiring precise racing line management and optimal exit speed. Sector Two features two long straights with both DRS zones (now overtake zones under 2026 rules), making it crucial for overtaking opportunities and top speed optimization. Sector Three is a technical section with hairpins and chicanes that demand precise braking and acceleration management.

Track Feature	Description
Circuit Length	5.281 km (3.281 miles)
Number of Turns	16 turns (9 right, 7 left)
DRS/Overtake Zones	2 zones (Sector Two straights)
Lap Record	1:22.109 (Max Verstappen, 2021)
Pit Lane Speed Limit	80 km/h in practice/qualifying, 60 km/h during race

3. Technology Stack & Software Architecture

3.1 Recommended Simulation Platforms

Selecting the right simulation platform is crucial for realistic physics and sensor simulation. Based on current research and open-source availability, the following platforms are recommended in order of suitability for F1-specific autonomous racing development.

Option A: Assetto Corsa with OpenAI Gym Interface (Recommended)

Assetto Corsa provides the most realistic racing physics among consumer simulators and has a well-developed OpenAI Gym interface (AssettoCorsaGym). This platform offers high-fidelity tire physics, aerodynamic simulation, and support for custom tracks including the Yas Marina Circuit. The OpenAI Gym wrapper enables seamless integration with reinforcement learning algorithms, making it ideal for training autonomous racing agents.

Option B: CARLA Simulator

CARLA is an open-source autonomous driving simulator built on Unreal Engine 4, offering excellent 3D sensor simulation including LiDAR, cameras, GNSS, and IMU. While primarily designed for urban autonomous driving, CARLA can be adapted for racing scenarios and provides comprehensive Python API support. The simulator excels at realistic sensor modeling and environmental conditions, making it suitable for developing perception systems alongside control algorithms.

Option C: F1/10 ROS Simulator

The F1/10 simulator is a ROS and Gazebo-based platform specifically designed for autonomous racing research. It mirrors the behavior of 1/10th scale autonomous race cars and includes support for LiDAR, IMU, and odometry sensors. While the physics are less realistic than Assetto Corsa, the platform offers excellent integration with ROS ecosystem and is well-documented for academic research purposes.

3.2 Core Software Components

Building a complete autonomous racing system requires multiple interconnected software components. The following table outlines the essential components, their purposes, and recommended implementations for your project.

Component	Purpose	Recommended Tools
RL Framework	Training neural network policies	Stable-Baselines3, RLlib, CleanRL
Physics Engine	Vehicle dynamics simulation	PyBullet, MuJoCo, Custom
Perception	Track boundary & obstacle detection	OpenCV, PyTorch, TensorFlow
Control System	Steering, throttle, brake control	MPC, PID, Pure Pursuit
Path Planning	Optimal racing line computation	Minimum Curvature, QP Solver

Component	Purpose	Recommended Tools
Telemetry	Performance analysis & logging	Custom Python, Plotly, Grafana

4. Development Phases with KILO

Using KILO as your AI coding assistant will significantly accelerate development. The following phases outline a structured approach to building your autonomous racing AI, with specific guidance on how to leverage KILO for each phase.

Phase 1: Environment Setup (Week 1-2)

The first phase establishes your development environment and ensures all components work together correctly. This foundational step is critical for smooth development in later phases.

1. Install Assetto Corsa (Steam version recommended) and download the Yas Marina Circuit mod. Ensure you have the correct version compatible with AssettoCorsaGym.
2. Clone the AssettoCorsaGym repository from GitHub and install Python dependencies (gymnasium, numpy, pytorch).
3. Configure the Python environment with CUDA support for your RTX 5070Ti. Install PyTorch with CUDA 12.x support.
4. Test the basic gym environment by running a simple random agent to verify all connections work correctly.
5. Set up version control (Git) and create a project structure for your codebase.

KILO Prompt Example: "Help me set up the AssettoCorsaGym environment. I have Assetto Corsa installed via Steam and Python 3.11. Create a requirements.txt file and a basic test script that verifies the connection."

Phase 2: Vehicle Dynamics Model (Week 2-4)

Understanding and modeling F1 car dynamics is essential for developing effective control strategies. This phase focuses on creating accurate vehicle dynamics models that capture tire behavior, aerodynamics, and powertrain characteristics specific to 2026 F1 regulations.

1. Research and document the 2026 F1 car specifications: weight (~768kg), power output (~1000hp combined), tire compounds, and aerodynamic characteristics.
2. Implement a tire model (Pacejka Magic Formula or brush tire model) that accurately represents tire forces under different loads, slip angles, and temperatures.
3. Create an aerodynamic model capturing downforce and drag as functions of speed and active aero settings.
4. Model the hybrid powertrain including energy harvesting (MGU-K, MGU-H if applicable) and deployment strategies.
5. Validate the dynamics model against real F1 telemetry data or high-fidelity simulation outputs.

KILO Prompt Example: "Create a Python class for F1 tire dynamics using the Pacejka Magic Formula. Include parameters for lateral force, longitudinal force, and combined slip. The model should handle tire temperature effects and degradation modeling."

Phase 3: Perception & State Estimation (Week 4-6)

The perception module provides the AI with awareness of its environment, including track boundaries, racing line position, and vehicle state. Accurate state estimation is crucial for making optimal control decisions.

1. Implement sensor simulation interfaces for cameras, LiDAR (if using CARLA), and IMU/GNSS sensors.
2. Create a track boundary detection system using cone detection (if simulating F1 track markings) or track map lookup.
3. Develop a state estimation module (Extended Kalman Filter or particle filter) to fuse sensor data and estimate position, velocity, and heading.
4. Build a racing line deviation calculator that measures distance from optimal line and predicts trajectory.
5. Create a lap time prediction model based on current state and remaining track segments.

Phase 4: Racing Line Optimization (Week 6-8)

Finding the optimal racing line is fundamental to achieving fast lap times. This phase focuses on computing and refining the racing line specifically for Yas Marina Circuit.

1. Obtain or create a detailed track map of Yas Marina Circuit with turn radii, banking, and surface characteristics.
2. Implement a minimum-curvature path optimization algorithm to find the geometric racing line.
3. Refine the racing line using minimum-time optimization (direct collocation or shooting methods) considering vehicle dynamics.
4. Segment the track into zones (braking, corner entry, apex, corner exit, acceleration) with specific control strategies for each.
5. Create multiple racing lines for different conditions (qualifying, race, wet weather) and energy management strategies.

KILO Prompt Example: "Implement a minimum-time optimal racing line solver using direct collocation. I have a track map in CSV format (x, y coordinates) and need to optimize for a vehicle with specific acceleration and braking limits. Use CasADi for the optimization."

Phase 5: Reinforcement Learning Agent (Week 8-12)

The RL agent learns optimal driving strategies through interaction with the simulation. This phase combines the previous components into a complete learning system.

1. Design the observation space: position, velocity, heading, steering angle, throttle, brake, gear, tire temperatures, energy store, distance to optimal line, upcoming curvature.
2. Design the action space: continuous steering (-1 to 1), throttle (0 to 1), brake (0 to 1), gear (-1 for downshift, 0 for maintain, 1 for upshift).
3. Create a reward function that balances lap time minimization, track limits compliance, tire management, and energy efficiency.

4. Implement the RL algorithm: Soft Actor-Critic (SAC) or Proximal Policy Optimization (PPO) are recommended for continuous control.
5. Set up parallel environment training using multiple simulation instances to maximize sample efficiency.
6. Implement curriculum learning: start with simplified physics, gradually increase realism and difficulty.
7. Use imitation learning from expert demonstrations (human drivers or optimal control solutions) to bootstrap training.

5. F1 Driver Knowledge Integration

A world-class F1 AI must embody the knowledge and decision-making capabilities of an expert driver. This section details how to encode critical racing knowledge into your AI system.

5.1 Gear Shift Strategy

Optimal gear shifting balances engine power delivery with acceleration requirements. The AI should shift up when engine RPM reaches the optimal shift point (typically around 12,000-15,000 RPM for F1 cars) where power begins to drop off. Shift down during braking to maintain engine RPM in the power band for corner exit. Implement short-shifting in certain corners to manage wheel spin on exit or to save fuel. The 8-speed gearbox in F1 requires precise gear selection for each corner, with higher gears used on straights and lower gears for tight corners.

5.2 Throttle & Brake Modulation

Throttle application must be progressive rather than sudden to prevent rear tire spin, especially in low-grip conditions or at corner exit. The AI should modulate throttle based on available grip, tire condition, and track surface. Brake application requires threshold braking (maximum brake pressure without locking wheels) for optimal deceleration. Trail braking (gradually releasing brake pressure while turning into a corner) helps rotate the car and carries more speed through corners. The brake-by-wire system in modern F1 cars requires integration with the hybrid system for energy recovery.

5.3 Corner Execution

Each corner requires a specific approach: braking point identification (using track markers, distances, or reference points), turn-in point (where to begin steering), apex identification (the closest point to the inside of the corner), and exit point (where to begin accelerating). The AI must adapt its line based on tire condition, fuel load, and energy deployment strategy. For high-speed corners, maintaining momentum is crucial, while for tight hairpins, late apex approaches often yield faster exits. The concept of 'slow in, fast out' applies to most corners, but optimal execution varies significantly by corner type and sequence.

5.4 Pit Stop Strategy

Pit stop decisions in F1 involve complex optimization considering tire degradation, track position, and race circumstances. The AI should monitor tire wear through temperature sensors and lap time degradation patterns. Determine optimal pit windows by calculating undercut/overcut potential

relative to competitors. Select appropriate tire compounds based on remaining laps, track conditions, and strategic requirements. The pit lane delta time (time lost by pitting compared to staying on track) must be factored into all strategic decisions. In a qualifying scenario, fresh tires and optimal tire preparation (warm-up laps) are critical for achieving fastest laps.

5.5 Energy Management (2026 Regulations)

The 2026 regulations introduce a 50:50 power split between internal combustion engine and electric motor, making energy management crucial. The AI must optimize energy harvesting during braking and lift-off phases. Deploy stored electrical energy strategically for acceleration out of corners and on straights. The new 'overtake mode' replaces DRS and provides extra electrical energy when within one second of another car. Energy deployment maps should be created for each track section, prioritizing acceleration zones where additional power provides the greatest lap time benefit. Active aerodynamics (switching between low-drag and high-downforce modes) must be coordinated with energy management for optimal performance.

6. Training Strategy & Hyperparameters

Effective training requires careful tuning of hyperparameters and a structured curriculum. With your hardware (RTX 5070Ti 12GB, 32GB RAM), you can efficiently train medium-sized networks while running multiple parallel environments.

6.1 Recommended Neural Network Architecture

For the policy network, use a multi-layer perceptron (MLP) with 3-4 hidden layers of 256-512 neurons each, or consider using a transformer-based architecture for better temporal reasoning. The actor network maps observations to action distributions (Gaussian for continuous actions), while the critic network estimates the value function. For processing sequential information (like upcoming track segments), consider adding LSTM or GRU layers to capture temporal dependencies. The network should output actions for steering, throttle, brake, and gear shift commands.

6.2 Hyperparameter Guidelines

Hyperparameter	Recommended Value	Notes
Learning Rate	3e-4 (actor), 1e-3 (critic)	Use decay schedule
Batch Size	256-512	Larger for stability
Discount Factor (γ)	0.99-0.999	High for lap completion
Parallel Environments	4-8 instances	Balance with VRAM
Entropy Coefficient	0.01 (initial)	Encourage exploration
Training Steps	10M-100M	Monitor convergence

7. Evaluation & Continuous Optimization

7.1 Performance Metrics

Track multiple metrics during training and evaluation: lap time (primary objective), sector times (identify weak sections), top speed achieved, average speed, tire temperature management, energy efficiency, track limit violations, stability metrics (steering smoothness, throttle consistency), and comparison to human benchmarks. Create visualization dashboards to monitor these metrics in real-time during training sessions.

7.2 Sim-to-Real Considerations

While your project targets virtual racing, the principles learned transfer to real-world applications. Domain randomization during training (varying track friction, car setup, weather conditions) improves robustness. System identification techniques help match simulation parameters to real-world behavior. Safety constraints and fallback behaviors should be implemented to handle edge cases and unexpected situations.

8. Effective KILO Usage for This Project

KILO can significantly accelerate your development process when used effectively. Here are specific strategies for leveraging KILO throughout your project.

- Break down complex tasks: Instead of asking KILO to 'build the RL agent', ask for specific components like 'create a replay buffer class with prioritized experience replay support'
- Provide context: Share relevant code snippets, error messages, and system specifications when asking for help
- Use iterative refinement: Start with a basic implementation, test it, then ask KILO to optimize specific aspects
- Request explanations: Ask KILO to explain complex algorithms or mathematical concepts to deepen your understanding
- Leverage debugging: When encountering errors, paste the full traceback and ask KILO to diagnose and fix the issue
- Code review: Ask KILO to review your implementations for potential bugs, performance issues, or best practice violations

9. Project Timeline & Milestones

Phase	Milestone	Duration
Phase 1	Environment setup, basic simulation running	Weeks 1-2
Phase 2	Vehicle dynamics model validated	Weeks 3-4
Phase 3	Perception system functional	Weeks 5-6

Phase	Milestone	Duration
Phase 4	Optimal racing line computed	Weeks 7-8
Phase 5	RL agent completing laps consistently	Weeks 9-10
Phase 6	Optimized agent approaching record times	Weeks 11-14

10. Key Resources & References

The following resources provide essential knowledge for your project. Regularly consult these references throughout development to ensure your AI adheres to regulations and implements best practices.

- FIA 2026 F1 Regulations Hub: https://www.fia.com/F126
- AssettoCorsaGym Repository: https://github.com/dasgringuen/assetto_corsa_gym
- CARLA Simulator: https://carla.org
- F1/10 Autonomous Racing: https://f1tenth.dev
- Stable-Baselines3 Documentation: https://stable-baselines3.readthedocs.io
- McLaren F1 Technical Specifications: https://www.mclaren.com/racing/formula-1/
- Yas Marina Circuit Information: https://www.racingcircuits.info/middle-east/united-arab-emirates/yas-marina.html

11. Getting Started Checklist

Before beginning development, ensure you have the following prerequisites in place:

- Assetto Corsa installed with Yas Marina Circuit mod
- Python 3.10+ with CUDA-enabled PyTorch
- Git repository initialized for version control
- Familiarity with KILO AI coding assistant
- Understanding of basic RL concepts (MDPs, policies, rewards)
- NVIDIA drivers updated for RTX 5070Ti

Best of luck with your F1 Autonomous Racing AI project!